

HDOC DAY : 34

AIM: To write a menu-driven C program using functions to verify the distributive properties of matrix multiplication over matrix addition:

1. Left Distributive Property: $A \times (B+C) = (A \times B) + (A \times C)$
2. Right Distributive Property: $(A+B) \times C = (A \times C) + (B \times C)$

ALGORITHM

1. Start.
2. Define functions to read, print, add, and multiply 2D matrices.
3. Display a menu with 3 options:
 - a. 1: Check Left Distributive Property [$A \times (B+C) = A \times B + A \times C$]
 - b. 2: Check Right Distributive Property [$(A+B) \times C = A \times C + B \times C$]
 - c. 3: Exit
4. If Option 1 is chosen:
 - a. Input dimensions m,n,p.
 - b. Input elements for matrices A, B, and C.
 - c. Calculate LHS = $A \times (B+C)$.
 - d. Calculate RHS = $(A \times B) + (A \times C)$.
 - e. Print LHS and RHS.
 - f. Compare LHS and RHS and print "Verified".
5. If Option 2 is chosen:
 - a. Input dimensions m,n,p.
 - b. Input elements for matrices A, B, and C.
 - c. Calculate LHS = $(A+B) \times C$.
 - d. Calculate RHS = $(A \times C) + (B \times C)$.
 - e. Print LHS and RHS.
 - f. Compare LHS and RHS and print "Verified".
6. If Option 3 is chosen, stop the program.

C CODE :

```
#include <stdio.h>

// Function to input a matrix
void getMatrix(int r, int c, int m[10][10]) {
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            scanf("%d", &m[i][j]);
        }
    }
}

// Function to print a matrix
void showMatrix(int r, int c, int m[10][10]) {
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }
}

// Function to add two matrices
void add(int r, int c, int m1[10][10], int m2[10][10], int res[10][10]) {
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            res[i][j] = m1[i][j] + m2[i][j];
        }
    }
}

// Function to multiply two matrices
void multiply(int r1, int c1, int c2, int m1[10][10], int m2[10][10], int res[10][10]) {
    int i, j, k;
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++) {
            res[i][j] = 0;
            for (k = 0; k < c1; k++) {
                res[i][j] += m1[i][k] * m2[k][j];
            }
        }
    }
}

int main() {
    int choice;
    int m, n, p;
    int A[10][10], B[10][10], C[10][10];
    int LHS[10][10], RHS[10][10];
    int temp1[10][10], temp2[10][10], temp3[10][10];

    while (1) {
        printf("\n1. Left Distributive Property\n");
    }
}
```

```

while (1) {
    printf("\n1. Left Distributive Property\n");
    printf("2. Right Distributive Property\n");
    printf("3. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Enter m, n, p: ");
        scanf("%d %d %d", &m, &n, &p);

        printf("Enter Matrix A:\n");
        getMatrix(m, n, A);

        printf("Enter Matrix B:\n");
        getMatrix(n, p, B);

        printf("Enter Matrix C:\n");
        getMatrix(n, p, C);

        // LHS = A * (B + C)
        add(n, p, B, C, temp1);
        multiply(m, n, p, A, temp1, LHS);

        // RHS = (A * B) + (A * C)
        multiply(m, n, p, A, B, temp2);
        multiply(m, n, p, A, C, temp3);
        add(m, p, temp2, temp3, RHS);

        printf("\nLHS Matrix:\n");

```

```

        printf("\nLHS Matrix:\n");
        showMatrix(m, p, LHS);

        printf("\nRHS Matrix:\n");
        showMatrix(m, p, RHS);

        printf("\nLeft Distributive Property is Verified!\n");
    } else if (choice == 2) {
        printf("Enter m, n, p: ");
        scanf("%d %d %d", &m, &n, &p);

        printf("Enter Matrix A:\n");
        getMatrix(m, n, A);

        printf("Enter Matrix B:\n");
        getMatrix(m, n, B);

        printf("Enter Matrix C:\n");
        getMatrix(n, p, C);

        // LHS = (A + B) * C
        add(m, n, A, B, temp1);
        multiply(m, n, p, temp1, C, LHS);

        // RHS = (A * C) + (B * C)
        multiply(m, n, p, A, C, temp2);
        multiply(m, n, p, B, C, temp3);
        add(m, p, temp2, temp3, RHS);

```

```
        // RHS = (A * C) + (B * C)
        multiply(m, n, p, A, C, temp2);
        multiply(m, n, p, B, C, temp3);
        add(m, p, temp2, temp3, RHS);

        printf("\nLHS Matrix:\n");
        showMatrix(m, p, LHS);

        printf("\nRHS Matrix:\n");
        showMatrix(m, p, RHS);

        printf("\nRight Distributive Property is Verified!\n");

    } else if (choice == 3) {
        printf("Exiting...\n");
        break;
    } else {
        printf("Invalid choice!\n");
    }
}

return 0;
}
```

OUTPUT SCREENSHOT :

```
[manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ % nano 34.c
[manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ % gcc 34.c -o 34
[manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ % ./34
```

1. Left Distributive Property
2. Right Distributive Property
3. Exit

Enter choice: 1

Enter m, n, p: 2 2 2

Enter Matrix A:

1 2

3 4

Enter Matrix B:

2 0

1 3

Enter Matrix C:

1 2

0 1

LHS Matrix:

5 10

13 22

RHS Matrix:

5 10

13 22

Left Distributive Property is Verified!

1. Left Distributive Property
2. Right Distributive Property
3. Exit

Enter choice: 2

Enter m, n, p: 2 2 2

Enter Matrix A:

1 2

3 4

Enter Matrix B:

0 1

2 3

Enter Matrix C:

2 1

1 0

LHS Matrix:

5 1

17 5

RHS Matrix:

5 1

17 5

Right Distributive Property is Verified!

1. Left Distributive Property
2. Right Distributive Property
3. Exit

Enter choice: 3

Exiting...

manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ %